

Encontro 1: Conceitos Básicos de Programação e Fluxogramas

Sumário

Encontro 1: Conceitos Básicos de Programação e Fluxogramas	1
1. Introdução à Lógica de Programação.....	2
2. Algoritmos	2
2.1. Tipos de Algoritmos	2
3. Pseudocódigo	2
4. Fluxograma	3
5. Programação por blocos	5
5.1. Como funcionam os algoritmos na programação por blocos:.....	5
5.2. Ferramentas baseadas em programação por blocos:.....	6
6. Comparação entre os tipos de algoritmos	7
7. Conclusão	7

1. Introdução à Lógica de Programação

A lógica de programação é um conjunto de regras e técnicas que permite a elaboração de sequências lógicas (algoritmos) para resolver problemas, decompondo-os em etapas menores e organizadas, que são traduzidas para uma linguagem que o computador pode entender e executar para realizar tarefas desejadas. É a base do desenvolvimento de software, focando no raciocínio estruturado para criar soluções previsíveis e eficientes.

2. Algoritmos

Um algoritmo é uma sequência finita de passos bem definidos que, quando executados, resolvem um problema ou executam uma tarefa. Pense em uma receita de bolo ou em um manual de instruções para montar um móvel. Ambos são exemplos de algoritmos do nosso dia a dia. Na computação, os algoritmos são a essência de qualquer programa. Eles precisam ser:

- **Finito:** Deve terminar após um número finito de passos.
- **Definido:** Cada passo deve ser claro e não deixar margem para interpretação.
- **Entrada:** Possui uma entrada de dados a partir da qual o algoritmo opera.
- **Saída:** Produz uma saída ou resultado após o processamento das informações.
- **Eficaz:** As instruções devem ser básicas o suficiente para serem executadas.

2.1. Tipos de Algoritmos

Algoritmo, portanto, pode ser entendido como sendo a representação do pensamento lógico como um roteiro detalhado ou um passo a passo para a solução de um problema, que pode ser expresso de diversas formas, como por meio de descrição narrativa, fluxograma ou pseudocódigo.

- **Descrição Narrativa:** Utiliza a linguagem natural para descrever os passos.
- **Pseudocódigo:** Utiliza uma linguagem intermediária, mais próxima da linguagem humana, mas com uma estrutura que se assemelha à de uma linguagem de programação.
- **Fluxograma:** Utiliza símbolos gráficos para representar o fluxo de execução.
- **Blocos:** Utiliza blocos que se encaixam para representar o fluxo de execução.

3. Pseudocódigo

O pseudocódigo é uma forma de escrever algoritmos que se assemelha a uma linguagem de programação, mas sem a rigidez da sintaxe. É uma ferramenta excelente para planejar a lógica de um programa antes de se preocupar com os detalhes de uma linguagem específica como JavaScript, Python ou C#. Ele permite que o desenvolvedor se concentre na lógica e na estrutura do algoritmo.

Exemplo de Pseudocódigo para somar dois números:

```
start
  read number1
  read number2
  sum = number1 + number2
  write The sum is:, sum
end
```

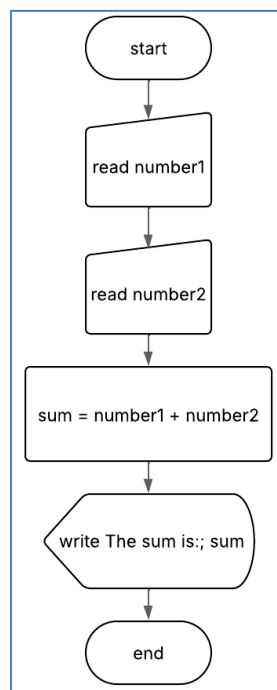
Exemplo de pseudocódigo para somar dois números.

4. Fluxograma

O fluxograma é uma forma visual de representar algoritmos e processos por meio de símbolos gráficos padronizados, como retângulos, losangos e setas. Diferente do pseudocódigo, que utiliza uma linguagem próxima à escrita, o fluxograma traduz a lógica de um programa em um diagrama que facilita a compreensão imediata da sequência de passos. Ele permite visualizar o fluxo de execução, as decisões e as repetições de forma clara e intuitiva, servindo como um mapa que guia o desenvolvedor e demais envolvidos no projeto.

Assim como o pseudocódigo, o fluxograma é uma ferramenta de planejamento, mas seu diferencial está no aspecto visual, o que favorece a comunicação entre pessoas de diferentes áreas, inclusive aquelas sem conhecimento técnico aprofundado em programação. Com ele, torna-se mais fácil identificar falhas, redundâncias ou melhorias no processo antes da implementação em linguagens específicas.

Exemplo de Fluxograma para somar dois números:



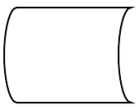
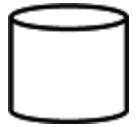
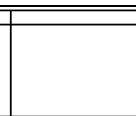
Exemplo de pseudocódigo para somar dois números.

Símbolos comumente usados em fluxogramas:

A tabela seguinte exhibe os símbolos mais frequentes em fluxogramas, cabendo, porém, uma importante observação: embora vários padrões para o uso de símbolos e a criação de fluxogramas tenham sido estabelecidos, não há problema em ignorá-los. Use os símbolos de uma forma que faça sentido para o seu público. Se você usar símbolos de forma não padronizada, certifique-se de fazê-lo consistentemente para que seus leitores entendam o significado do símbolo sempre que o virem.

Conceitos Básicos de Programação e Fluxogramas

Símbolo	Imagem	Significado
Terminador (Início/Fim)		Pontos iniciais, finais e potenciais resultados de um caminho. Geralmente contém "Início" ou "Fim" dentro da forma.
Processo		Processo, ação ou função. É o símbolo mais utilizado em fluxogramas.
Processo predefinido		Processo ou operação complicada que é definida em outro lugar.
Decisão		Pergunta a ser respondida (sim/não, verdadeiro/falso). O caminho do fluxograma pode então se dividir em diferentes ramificações, dependendo da resposta.
Seta		Mostra a direção do fluxo de execução.
Comentário / nota		Adiciona explicações ou comentários necessários ao intervalo especificado. Pode também ser conectado por uma linha tracejada à seção relevante do fluxograma.
Conector		Usados em gráficos mais complexos, este símbolo conecta elementos separados na mesma página.
Conector externo		Conecta elementos de outras páginas, com o número da página colocado ao lado ou dentro da forma para fácil referência.
Separador de fluxo		Indica que o fluxo do processo continua em dois ou mais caminhos.
Mesclagem de fluxo		Combina vários caminhos para se tornarem um.
Entrada/Saída		Dados disponíveis para entrada ou saída, bem como recursos utilizados ou gerados.
Exibição em tela		Informações a serem exibidas na tela do computador.
Entrada manual		Entrada manual de dados, geralmente por teclado ou dispositivo. Um exemplo seria a necessidade do usuário inserir dados manualmente.
Operação Manual		Etapa que deve ser realizada manualmente, não automaticamente.
Documentos		Entrada ou saída de um documento. Entrada: recebimento de relatório, e-mail ou pedido. Saída: geração de apresentação, memorando ou carta.
		Idem ao anterior, para vários documentos ou relatórios.

Símbolo	Imagem	Significado
Armazenamento de dados		Representa genérica de dados armazenados dentro de um processo.
		Dados guardados em um serviço de armazenamento que provavelmente permitirá pesquisa e filtragem por usuários. (memória, banco de dados).
		Indica dados armazenados em disco rígido, também conhecido como armazenamento de acesso direto.
		Comumente usados para mapear projetos de software, esse formato indica dados armazenados na memória interna.

5. Programação por blocos

A programação por blocos é uma abordagem visual para o desenvolvimento de algoritmos, na qual os comandos são representados por blocos gráficos que podem ser encaixados como peças de um quebra-cabeça. Em vez de digitar linhas de código em uma linguagem textual, o programador monta sequências lógicas arrastando e conectando blocos que simbolizam instruções, condições, repetições e variáveis.

Essa forma de programar é especialmente voltada para iniciantes, pois elimina a preocupação com erros de sintaxe e torna o aprendizado mais acessível e intuitivo. Além disso, o aspecto visual ajuda a compreender melhor conceitos fundamentais de programação, como sequência, decisão e repetição. Plataformas como Scratch, Blockly e App Inventor popularizaram essa abordagem, permitindo que crianças, estudantes e iniciantes criem jogos, animações, aplicativos e até robôs.

A grande vantagem da programação por blocos é possibilitar que o estudante se concentre na lógica e na estrutura dos algoritmos, preparando-o para migrar futuramente para linguagens de programação tradicionais.

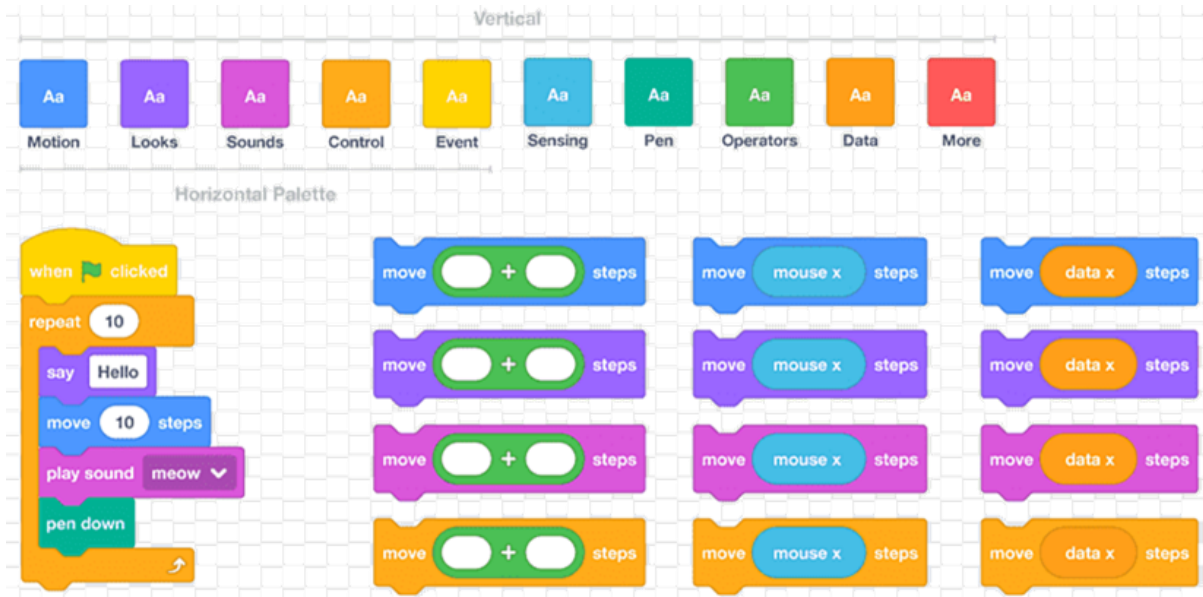
Na programação por blocos, os algoritmos são criados juntamente com a programação por blocos visuais e interativos, que permitem a criação de animações, jogos e histórias. Os usuários arrastam e soltam esses blocos para construir a sequência de instruções, que são testadas e executadas na própria plataforma. Essa abordagem visual e interativa facilita a criação de lógicas de programação complexas, como repetições, condicionais e uso de variáveis, de maneira intuitiva e acessível para iniciantes.

5.1. Como funcionam os algoritmos na programação por blocos:

- Interface de Blocos: são usados blocos de diferentes cores e formas que representam comandos de programação.
- Construção de Lógica: para criar um algoritmo, os usuários combinam esses blocos em sequências, que podem incluir instruções como movimento, aparência, som, eventos e variáveis.
- Exemplos de Blocos:
 - Movimento: Bloquinhos para fazer um personagem andar, girar ou mudar de posição.

Conceitos Básicos de Programação e Fluxogramas

- Controle: Blocos para repetições (como "repita 5 vezes" ou "sempre") e para criar condições (comandos "se" e "senão").
- Variáveis: Blocos para armazenar e modificar informações, como números ou texto, que podem mudar durante a execução do programa.
- Execução e Teste: Ao clicar no botão de execução, o algoritmo é executado, permitindo que o usuário veja os resultados das instruções e faça os ajustes necessários.



Exemplo de algoritmo implementado em Scratch e lista parcial dos blocos disponíveis.

5.2. Ferramentas baseadas em programação por blocos:

- Para jogos e lógica básica:
 - Scratch — <https://scratch.mit.edu/>
 - Snap! — <https://snap.berkeley.edu/>
 - Code.org — <https://code.org/>
- Para apps móveis:
 - App Inventor — <https://appinventor.mit.edu/>
- Para robótica e eletrônica:
 - mBlock — <https://mblock.makeblock.com/>
 - Tinkercad — <https://www.tinkercad.com/>
- Para personalizar em projetos próprios:
 - Blockly — <https://developers.google.com/blockly>

6. Comparação entre os tipos de algoritmos

Aspecto	Pseudocódigo	Fluxograma	Programação por Blocos
Forma de Representação	Texto estruturado semelhante a comandos de programação.	Símbolos gráficos conectados em um diagrama.	Blocos coloridos que se encaixam como peças de quebra-cabeça.
Foco	Estrutural e textual, com ênfase em planejar a lógica sem se preocupar com sintaxe real.	Visual e esquemático, destacando o fluxo de execução do algoritmo.	Visual, intuitivo, interativo, ênfase em explorar a lógica de forma prática.
Complexidade	Exige compreensão básica de estruturas de algoritmos, mas sem linguagem formal.	Exige conhecer símbolos padrão, mas é fácil de interpretar.	Muito acessível para iniciantes, não exige escrita de código.
Aprendizado	Útil para estudantes que se preparam para linguagens reais.	Útil para comunicação visual entre pessoas técnicas e não técnicas.	Ideal para introduzir programação em crianças e iniciantes.
Exemplos de Uso	Planejamento de programas em C#, Python, JavaScript.	Mapeamento de processos, lógica de programas, fluxos em empresas.	Jogos, animações, histórias interativas, simulações simples.
Vantagem Principal	Clareza lógica e proximidade das linguagens reais.	Clareza visual, facilita identificar falhas ou redundâncias no processo.	Aprendizado lúdico, sem erros de sintaxe.

7. Conclusão

A compreensão da lógica de programação e de suas diferentes formas de representação — seja por meio de pseudocódigo, fluxogramas ou programação por blocos — constitui a base essencial para o desenvolvimento de qualquer aplicação na área de tecnologia. Nesta apostila, buscou-se apresentar esses conceitos de maneira gradual, permitindo que o estudante desenvolva a capacidade de estruturar soluções de forma clara, lógica e eficiente.

No contexto do curso de Programador Web, esse aprendizado inicial é especialmente relevante. Antes de trabalhar com linguagens como HTML, CSS, JavaScript ou frameworks modernos, é fundamental dominar a lógica que sustenta os algoritmos e a organização dos processos computacionais. Assim, o estudante passa a compreender que a criação de páginas dinâmicas, sistemas de autenticação, manipulação de dados ou mesmo a integração com bancos de dados são construções que dependem da mesma base: raciocínio lógico estruturado.

Mais do que ferramentas, os conteúdos apresentados aqui são ferramentas cognitivas. Eles permitem que o futuro programador não apenas reproduza códigos prontos, mas consiga analisar problemas, propor soluções criativas e implementar sistemas que sejam funcionais e eficientes. Portanto, este material serve como alicerce para a jornada do estudante rumo ao desenvolvimento web, conectando a teoria da lógica de programação às práticas que o mercado exige de um profissional da área.